NarraCare: Final Presentation Script

English main talk: 299 words. Target: 3 minutes including brief pauses. Read only the body paragraphs aloud. Slides 7-9 remain Q&A backups.

Slide 1 | 0:00-0:12 | NarraCare

NarraCare is a conversational rehabilitation recommender with a gentle trail companion. It combines an evaluated recommendation pipeline with a live LLM agent.

Slide 2 | 0:12-0:35 | The problem and our setting

Home rehabilitation can feel repetitive, while symptoms change daily. We want suitable recommendations and manageable achievements, without pressure to push through discomfort. Our benchmark uses twenty synthetic personas, one hundred exercises and sixty-day rollouts, covering knee, shoulder and chronic back conditions.

Slide 3 | 0:35-1:20 | How recommendation works

Our core is a contextual bandit recommender. Patient state and interaction history form the context. Safety restricts the candidates, and SAPS ranks them using LinUCB's reward estimate and exploration bonus, plus a sampled Beta safety bonus. Reported outcomes update the reward and safety statistics. In the benchmark, feedback comes from the simulator. In the live agent, it comes from the user. The LLM interprets requests and selects guarded tools; ranking and safety checks remain in Python.

Main talk, continued

Slide 4 | 1:20-2:00 | What the results show

SAPS earns forty-two point zero two, close to LinUCB, without a significant advantage. Random earns more reward, but its infeasible proposal rate is forty point eight two percent, versus SAPS's two point two three percent. IER checks ungated proposals; SVR checks accepted tasks. Removing Safety from LinUCB produces one hundred seventy-one violating accepted tasks. These are benchmark results, not an evaluation of conversation quality.

Slide 5 | 2:00-2:40 | The interactive companion

The companion turns rehabilitation into a mountain trail, with small milestones and camp when tired. In our verified Gradio session, changing a preference changed the actual candidate set and task. Explicit completion updated the learner; skipping created camp without invented feedback. These checks demonstrate integration. Whether the companion improves engagement still requires a user study.

Slide 6 | 2:40-3:00 | Contribution and next steps

Our contribution is an inspectable framework for constrained online recommendation, with separate measurements of proposals and accepted actions. The agent makes this workflow interactive. Further work must isolate the added learning mechanisms and validate the data and user experience. Thank you.

Rehearsal cues: finish slide 3 by 1:20 and slide 4 by 2:00. Use the existing screenshot on slide 5. Do not add an unrehearsed live run inside the three-minute slot.

Q&A Backup

Not part of the three-minute talk. Answer only the question asked; each response is approximately 15-30 seconds.

What dataset did you use, and where did it come from?

We use an authored synthetic dataset: twenty personas, one hundred tasks, and fifty-one design-time safety definitions. Five pre-filter predicates run in the main safety path. This is a controlled simulation, not Kaggle data or patient records. The tasks and clinical annotations still need independent clinical validation.

What is the recommendation technology, and what is SAPS?

We use contextual bandits that learn online from feedback. SAPS adapts LinUCB by adding a sampled Beta safety bonus based on whether reported pain increases by at most one point. This is a task-specific adaptation, not a new bandit theory. A separate hard filter enforces the implemented restrictions.

Why not just use Random?

Random has higher simulator reward, and the shared Safety filter protects both policies. SAPS has lower ungated infeasible proposals, but does not significantly outperform LinUCB. We therefore claim an inspectable constrained-recommendation framework, not algorithmic superiority.

How do users provide state, and how does feedback change recommendations?

The current demo starts from a selected synthetic profile. Users report symptoms and preferences in English chat; the agent asks for missing information. Preference changes affect retrieval. Reported completion and pain outcomes update the learner. Satisfaction is collected but currently does not enter the reward formula. There is no medical-record import.

What makes this an agent rather than just a chatbot?

GPT-four-o-mini interprets requests and chooses guarded tools for clarification, retrieval, ranking and feedback. Python tools enforce state requirements and task restrictions. The LLM does not directly invent task scores or awards. Saying I will try is not treated as completion.

Without real patient data, what is useful and what is unproven?

The experiments test behavior under specified simulated conditions. For example, removing Safety increases rule-violating accepted tasks. This supports the filtering mechanism within our model, not real-world clinical safety. The reusable parts are the component interfaces and proposal-versus-execution diagnostics. Clinical benefit and engagement remain unproven.